AI APIs as Data Tools

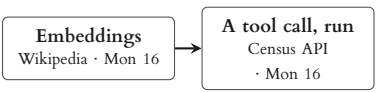
Week 13 · APIs module · Textbook Chapter 13

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

November 9, 13 & 16, 2026

This week at a glance





Monday, Nov 9 | Part 1: a PDF becomes Markdown
Wednesday, Nov 11 | No class **Friday, Nov 13** | Part 2: a model returns JSON Then 15 minutes on portfolio checkpoint 3.
Monday, Nov 16 | Part 3: embeddings, and one tool call

Keys

`DATALAB_KEY` and `OPENROUTER_KEY`. Part 3 also uses week 11's `CENSUS_KEY`.

Due

Sunday, Nov 15: portfolio checkpoint 3.
Sunday, Nov 22: chapter 13's Recommended Exercises.

1. Monday · Live-Coding Lab

This week

Find it | `labs/week-13` in the course repository

1. Open `lab-13.ipynb` and click **Download raw file**.
2. Open it in JupyterLab, in the `webdata` environment.

Use it | one part a day

- I type the code on screen. You type it in your copy.
- Each part starts at its own setup step.

Fall behind?

Copy the cell from `lab-13-key.ipynb`, beside it, and keep going.

Errors

A `429` means a free model is busy: wait, and send it again.

A model called like a function

A chatbot answers a person. This week a model answers a program.

We send a request and get data back, as with week 11's APIs. Then we check that data against something we trust.

Who sees what you send

Every page goes to Datalab, or to OpenRouter and the model's owner. Send nothing you couldn't publish.

Step 0: store two keys

Get a key at datalab.to and at openrouter.ai.

Store both in a terminal. Then close it, open a new one, and start Jupyter from there.

```
# Store each key in the webdata environment
conda env config vars set DATALAB_KEY="..." -n webdata
conda env config vars set OPENROUTER_KEY="..." -n webdata
```

Step 0: check them

Print whether each key is set. Never print the key itself.

Does yours say `MISSING`? Run the given `getpass()` cell under it.

The notebook writes each check in one line.

```
# Is the Datalab key set? Print "set" or "MISSING"
found = os.environ.get("DATALAB_KEY")
print("DATALAB_KEY:", "set" if found else "MISSING")
# The same for the OpenRouter key
found = os.environ.get("OPENROUTER_KEY")
print("OPENROUTER_KEY:", "set" if found else "MISSING")
```

Step 1: what to send

We send Datalab a link to week 9's 2024 report, not the file. Datalab fetches the PDF itself.

Which line asks for Markdown? Which asks for the cost of a fresh conversion?

Docs: Datalab's Convert API

```
# A link to the PDF, and how to convert it
data = {
    "file_url": book + "revenue-report-2024-12.pdf",
    "output_format": "markdown",
    "mode": "balanced",
    "paginate": "true",
    "skip_cache": "true",
}
```

Step 1: submit it

Post it with your key in a header.

What comes back first is a ticket.
Which field is it?

```
# Send it, with the key in a header
job = requests.post(
    f"{datalab_base}/convert",
    data=data,
    headers={"X-API-Key": datalab_key, **HEADERS},
).json()
# The ticket
print(job)
```

Step 2: ask about the ticket

The ticket, `request_check_url`, is where the result will appear. Ask it once.

`processing` means Datalab is still reading.

```
# Ask the ticket's address how the job is going
check = requests.get(
    job["request_check_url"],
    headers={"X-API-Key": datalab_key, **HEADERS},
).json()
print(check["status"])
```

Step 2: wait for the result

This cell is given. It asks again every two seconds until the job is done, then prints the pages and the cost.

Discuss: what would this loop do if Datalab never finished?

```
# Ask again every two seconds while it's processing
while check["status"] == "processing":
    time.sleep(2)
    check = requests.get(
        job["request_check_url"],
        headers={"X-API-Key": datalab_key, **HEADERS},
    ).json()
# Done: print the status
print("status:", check["status"])
```

Step 3: split the Markdown into pages

Each page starts with a marker such as `{14}` and a line of dashes. Split on the markers, and drop the empty piece before the first one.

Never print `pages[0]`: the cover gives a staff member's phone and email.

```
# The whole report, as one string of Markdown
markdown = check["markdown"]
# Split it at each page marker; drop the empty first piece
pages = re.split(r"\n*\{\d+\}-{10,}\n*", markdown)[1:]
# How many pages, and the start of the summary page
print(len(pages), "pages")
print(pages[1][:1200])
```

Step 3: a picture of a table

Page 15, Exhibit 3, is an image. Week 9's pdfplumber found no text in it.

Print the first lines of the Markdown table Datalab wrote. Does it look like the PDF?

```
# Page 15 (index 14), from the table's first "|"
start = pages[14].index("|")
exhibit = pages[14][start:]
# The first 10 lines, each cut at 150 characters
for line in exhibit.splitlines()[:10]:
    print(line[:150])
```

What Datalab read, and what its model wrote

Datalab **read** Chart 8's figures from its slices. Its model also **wrote** a sentence about them.

The sentence calls $1,262,688 the largest slice. Is it?

Discuss: how would you tell what was read from what was written?

Chart 8, page 9	Amount
State Share-Back	$529,220
Medical Marijuana (3.86% Base)	$52,937
Recreational Marijuana (3.86% Base)	$1,262,688
Recreational Marijuana (3.5% + 5.0% Excise)	$1,341,007

Step 4: split each row into cells

Week 9's first check: a row's twelve months should add up to its total.

First, split each line on | and clean each cell. The notebook does the cleaning in one list comprehension.

```
for line in pages[14].splitlines():
    # Split the row at each "|", and clean each cell
    cells = []
    for c in line.split("|"):
        c = c.strip().replace(",", "")
        c = c.replace("<b>", "").replace("</b>", "")
        cells.append(c)
```

Step 4: keep the rows with a year

A row we want has more than 16 cells, and its third cell is a year.

Why check the length first?

```
for line in pages[14].splitlines():
    ... # split and clean, as on the last frame
    # Keep a row with enough cells
    if len(cells) > 16:
        # ...whose third cell is a year
        if re.fullmatch(r"20\d\d", cells[2]):
            ... # add up its months: next frame
```

Step 4: add up the months

Add the twelve months, and compare the sum with the row's total. Print a row only if it misses by more than $5.

Nothing printed? What can a sum still miss?

```
checked = 0
for line in pages[14].splitlines():
    ... # split, clean, and keep the year rows
            # Add the months; compare with the total
            months = sum(int(c) for c in cells[3:15])
            checked += 1
            if abs(months - int(cells[15])) > 5:
                print(cells[2], months, cells[15])
print(checked, "rows checked")
```

2. Wednesday

I was so very very sick



3. Friday · Live-Coding Lab

Today: Part 2, a model as a function

We will use OpenRouter to call a free language model through an API. We send an instruction and one page of a report, and get back JSON.

- Find the page with the summary table's last row
- Ask for its totals as JSON
- This requires us to define the fields we want and their types
- Check them against `pdfplumber`'s figures; then do all seven reports

Start at Step 6

Download the new `lab-13.ipynb`. Today needs only `OPENROUTER_KEY`.

Free models are shared

The whole class uses the same free models. A busy one answers `429`.

Step 6: set up again

Put your email in `HEADERS`, and check the key.

Today we read the book's saved Markdown of all seven reports, so no one pays Datalab again.

```
# Is the key set?
found = os.environ.get("OPENROUTER_KEY")
print("OPENROUTER_KEY:", "set" if found else "MISSING")
# Where OpenRouter's API is, and the book's saved Markdown
openrouter_base = "https://openrouter.ai/api/v1"
saved = (
    "https://raw.githubusercontent.com/cuinfoscience/"
    "Web-Data-Science-Book/main/data/ch-13/"
)
```

Step 7: find the summary page

The model sees only what we send. We send one page: the one with the row *Total Sales and Use Tax*.

Read the start of that page. Where is the row we want?

```
# Read the 2024 report's saved Markdown
report = saved + "revenue-report-2024-12.md"
markdown = requests.get(report, headers=HEADERS).text
# Split it into pages, as on Monday
pages = re.split(r"\n*\{\d+\}-{10,}\n*", markdown)[1:]
# The first page with the row we want
matches = [page for page in pages
           if "Total Sales and Use Tax" in page]
summary = matches[0]
print(summary[:1000])
```

Step 8: the shape of the answer

A **JSON Schema** names the fields we want and the type of each.

This cell is given. In the notebook, every field also has a description, and `MONTHS` lists the twelve months.

```
# The answer's shape: four fields, each with a type
SCHEMA = {
    "type": "object",
    "properties": {
        "month": {"type": "string", "enum": MONTHS},
        "year": {"type": "integer"},
        "total_prior": {"type": "integer"},
        "total_current": {"type": "integer"},
    },
    "required": [...], # all four
}
```

Step 8: the instruction

The instruction tells the model its job. It is also given.

Which words in it try to stop the model from guessing?

```
# The model's job, in plain words
INSTRUCTION = (
    "You extract figures from one page of a City of "
    "Boulder revenue report. Copy numbers exactly as "
    "printed, without $ or commas. Use only the page."
)
```

Step 8: three models

The whole class shares these free models. If the first is busy, OpenRouter tries the next.

```
# Three free models, tried in this order
MODELS = [
    "nvidia/nemotron-3.5-super-vl:free",
    "google/gemma-4-31b-it:free",
    "dots-studio/dots-3-note-preview:free",
]
```

Step 8: the request's body

`temperature` 0 asks for the model's most likely answer.

The next two frames fill in the . . . .

```
# What we send to the model
body = {
    "models": MODELS,
    "temperature": 0,
    "messages": [...],
    "response_format": {...},
    "provider": {"require_parameters": True},
}
```

Step 8: what the model reads

Two messages. The `system` message is our instruction. The `user` message is the page.

There is no conversation: one request, one answer.

```
# First our instruction, then the page
"messages": [
    {"role": "system", "content": INSTRUCTION},
    {"role": "user", "content": summary},
],
```

Step 8: what the model must return

`response_format` asks for JSON in `SCHEMA`'s shape. `strict`: no other fields.

`require_parameters` sends the request only to models that accept this.

Docs: OpenRouter's structured outputs

```
# Answer in SCHEMA's shape, with no other fields
"response_format": {
    "type": "json_schema",
    "json_schema": {
        "name": "totals",
        "strict": True,
        "schema": SCHEMA,
    },
},
# Only models that accept response_format
"provider": {"require_parameters": True},
```

Step 8: send it

Post the body to the chat completions endpoint, with your key in the header.

```
# Post the body, with the key in a header
response = requests.post(
    f"{openrouter_base}/chat/completions",
    json=body,
    headers={
        "Authorization": f"Bearer {openrouter_key}",
        **HEADERS,
    },
).json()
```

Step 8: what came back

Print three things: any error, which model answered, and `usage`, the tokens in and out.

Did the first model answer?

```
# Any error?
print(response.get("error"))
# Which model answered, and was it one of ours?
print(response["model"], response["model"] in MODELS)
# How many tokens in, and how many out
print(response["usage"])
```

Step 8: read the answer

The answer arrives as text that holds JSON. `json.loads()` turns it into a dictionary.

Look at your four fields. Which would you trust without checking?

```
# The first choice's message holds the answer, as text
content = response["choices"][0]["message"]["content"]
# Turn that text into a dictionary
answer = json.loads(content)
print(answer)
```

Step 9: check the prior year

`TRUTH`, given, holds week 9's pdfplumber figures for each report. That is our **ground truth**.

Does the model's total for 2023 match?

```
# Week 9's two totals for the 2024 report
prior, current = TRUTH["revenue-report-2024-12"]
# The model's total, week 9's, and whether they match
print(
    "prior: ",
    answer["total_prior"],
    prior,
    answer["total_prior"] == prior,
)
```

Step 9: check the report's year

The same for 2024's total. Then look at the year and month.

Is the year 2024? Where on the page could another year come from?

```
# The same for the report's own year
print(
    "current:",
    answer["total_current"],
    current,
    answer["total_current"] == current,
)
# Is this 2024, and December?
print(answer["year"], answer["month"])
```

Microsprint: the same request, a different model

Send the same body to the third model alone.

Does it return the same four fields? Does your neighbor's run agree with yours?

```
# The same body, but only the third model in the list
response = requests.post(
    f"{openrouter_base}/chat/completions",
    json={**body, "models": [MODELS[2]]},
    headers=..., # your key, as above
).json()
# Which model answered, and what it said
content = response["choices"][0]["message"]["content"]
print(response["model"], json.loads(content))
```

Same request, different answers

Run Step 8 twice, and the answer can change. We have seen the year, and once both totals, change between runs.

`temperature` 0 asks for the most likely answer. The model still runs on shared machines, and small differences in the arithmetic can change which answer comes out.

So record

the model that answered, the date, and the raw response.

Step 10: Step 8 as a function

We will send Step 8's request seven times, once for each report. Only the page changes.

So the body goes in a function that takes the page.

```
def extract_totals(page):
    # Step 8's body, with this page as the user message
    body = {
        "models": MODELS,
        "temperature": 0,
        "messages": [
            {"role": "system", "content": INSTRUCTION},
            {"role": "user", "content": page},
        ],
        ... # response_format and provider, as before
    }
```

Step 10: send, and return the answer

The rest of the function sends the body and returns two things: the answer, and the model that gave it.

```
def extract_totals(page):
    ... # the body, as on the last frame
    # Send it, as in Step 8
    response = requests.post(
        f"{openrouter_base}/chat/completions",
        json=body,
        headers=...,
    ).json()
    # The answer, and the model that gave it
    message = response["choices"][0]["message"]
    answer = json.loads(message["content"])
    return answer, response["model"]
```

Step 10: seven reports

For each report, read its Markdown, find its summary page, and send it.

```
rows = []
for report, (prior, current) in TRUTH.items():
    # Read this report, and split it into pages
    md = requests.get(
        saved + report + ".md",
        headers=HEADERS,
    ).text
    pages = re.split(r"\n*\{\d+\}-{10,}\n*", md)[1:]
    # Send its summary page to the model
    matches = [page for page in pages
               if "Total Sales and Use Tax" in page]
    answer, model = extract_totals(matches[0])
```

Step 10: record each answer, and check it

Each row keeps the model's answer and whether each total matches week 9.

`time.sleep(2)` spaces out the requests to a shared free model.

```
# Keep the answer, and whether each total matches
rows.append({
    "report": report,
    "model": model,
    **answer,
    "prior_ok": answer["total_prior"] == prior,
    "current_ok": answer["total_current"] == current,
})
time.sleep(2)
```

Step 10: the table

Which reports came back wrong in your table? Are they the ones your neighbor got wrong?

```
# One row per report
results = pd.DataFrame(rows)
# Print it without the model column
print(results.drop(columns="model").to_string())
```

One way it goes wrong

The 2019 summary ends with two total rows. The last one is the year's total.

If your 2019 is wrong, compare its numbers with these rows. Which did the model copy?

	2018	2019
Total … current economic activity	$139,292,879	$139,548,151
Total Sales and Use Tax	$140,609,751	$141,954,650

Step 10: the schema, in the instruction

Add the schema to the instruction, as text. A given cell runs the loop again.

Compare the two tables. Was the problem the model, or what we sent it?

```
# Add the schema to the instruction, as text
INSTRUCTION = (
    INSTRUCTION
    + " Answer with JSON that fits this schema: "
    + json.dumps(SCHEMA)
)
```

Friday's last 15 minutes: portfolio checkpoint 3

Due Sunday, Nov 15 | Analyze your data

- **An analysis notebook** that runs top to bottom, with its output visible
- **Results that answer at least part of your question**: the counts or comparisons your proposal named
- **At least one figure** saved in your repository, with a title, labeled axes, and a one-sentence caption
- **The README's Methods section** and your preliminary **Findings**

Run your analysis notebook once, top to bottom, here, where you can ask. Used a model anywhere? Say in Methods which one, when, and what you checked its output against.

4. Monday, Nov 16 · Live-Coding Lab

Today: Part 3, search by meaning, and tools

On Friday the model read one page that we chose. With a hundred passages, how do we find the ones worth reading?

An **embedding** turns a passage into a list of numbers. Passages about the same thing get similar numbers.

Start at Step 11

Download the new `lab-13.ipynb`. Today uses `OPENROUTER_KEY` and week 11's `CENSUS_KEY`.

Five articles

Obama, Hillary Clinton, Trump, Biden and Harris, on January 1 of each year, 2004–2026.

Step 11's first two cells are given. Put your email in `HEADERS`, then run both.

Both keys should say `set`. If one says `MISSING`, the second cell asks for it.

The lead

The paragraphs above an article's first heading. Its job is to summarize the article.

Step 11: ask for one old version

Which revision of Obama's article did a reader see on January 1, 2009? Start at that moment, walk back in time, and stop after one.

Docs: the Action API's revisions

```
# One revision: the last one before 2009 began
params = {
    "action": "query",
    "titles": "Barack Obama",
    "prop": "revisions",
    "rvprop": "ids|timestamp",
    "rvlimit": 1,
    "rvstart": "2009-01-01T00:00:00Z",
    "rvdir": "older",
    "format": "json",
}
```

Step 11: find one old version

Send it, then go down to the page and its one revision.

What is the revision's timestamp? Why is it before midnight?

```
# Send it
data = requests.get(
    API,
    params=params,
    headers=HEADERS,
).json()
# Go down to the page, then to its one revision
found = data["query"]["pages"]
page = list(found.values())[0]
revision = page["revisions"][0]
print(revision)
```

Step 11: ask for its lead

`action=parse` returns a revision as HTML. `section=0` asks for the lead alone.

Docs: parse

```
# That revision's lead (section 0), as HTML
params = {
    "action": "parse",
    "oldid": revision["revid"],
    "prop": "text",
    "section": 0,
    "format": "json",
    "formatversion": 2,
}
```

Step 11: get its lead as HTML

Read the paragraphs. Does anything look out of place?

```
# Send it; the HTML is under "parse", then "text"
html = requests.get(
    API,
    params=params,
    headers=HEADERS,
).json()["parse"]["text"]
# Parse it, and print the start of each paragraph
soup = BeautifulSoup(html, "html.parser")
for paragraph in soup.find_all("p"):
    print(paragraph.get_text(" ", strip=True)[:150])
```

Step 11: delete the leftovers

Wikipedia draws an old version with today's templates. Deleted templates come back as stray links.

This given cell finds them by CSS selector, with the tables, footnote marks and error messages, and deletes each.

```
# Find the leftovers by CSS selector
leftovers = soup.select(
    "table, .reference, .hatnote, .shortdescription, "
    "style, a.new[title^='Template:'], .error"
)
# Delete each one from the page
for tag in leftovers:
    tag.decompose()
```

Step 11: join the paragraphs

Keep each paragraph that has text, and join them with blank lines.

The notebook does the join and the clean-up in one line.

```
# Keep the text of each paragraph that has any
paragraphs = []
for paragraph in soup.find_all("p"):
    if paragraph.get_text(strip=True):
        text = paragraph.get_text(" ", strip=True)
        paragraphs.append(text)
# Join them; drop the space before punctuation
joined = "\n\n".join(paragraphs)
lead = re.sub(r" ([,.;:])", r"\1", joined)
```

Step 11: the saved leads

A given cell loads all 116 leads, collected and cleaned this way on November 9.

Is your 2009 lead the same as the saved one?

```
# The saved lead for Obama's article in 2009
is_obama = leads["title"] == "Barack Obama"
is_2009 = leads["year"] == "2009"
saved_lead = leads[is_obama & is_2009]["lead"].iloc[0]
# Is it the same as the one you just cleaned?
print(lead == saved_lead)
```

Step 12: embed one lead

Send one lead to the embeddings endpoint. It comes back as a **vector**, a list of numbers.

How many numbers? No single one means anything. What matters is how two vectors compare.

```
EMBEDDER = "nvidia/nemotron-3-embed-1b:free"
# Send the first lead to the embeddings endpoint
one = requests.post(
    f"{openrouter_base}/embeddings",
    json={
        "model": EMBEDDER,
        "input": [leads["lead"][0]],
    },
    headers=..., # your key, as on Friday
).json()
# How many numbers came back?
print(len(one["data"][0]["embedding"]), "numbers")
```

Step 12: embed them all

A given cell embeds every lead, 20 to a request, and scales each vector to length 1.

It uses one model and no fallback list: two models' vectors can't be compared. If the model's daily limit stops you, the next given cell loads the vectors saved on November 9.

Cosine similarity

How nearly two vectors point the same way: 1 if exactly. For vectors of length 1, it is their dot product, `@`.

Step 12: compare each lead with today's

Keep each article's current vector.
Then compare every lead with its own article's current lead.

```
# Each article's current vector
today = {}
for i, row in leads.iterrows():
    if row["year"] == "current":
        today[row["title"]] = vectors[i]
# Each lead against its own article's current lead
leads["similarity"] = [
    vectors[i] @ today[title]
    for i, title in enumerate(leads["title"])
]
```

Step 12: check the current rows

A current lead compared with itself should give 1.

Do all five?

```
# The five current leads
current = leads[leads["year"] == "current"]
# Each one's similarity with itself
print(current[["title", "similarity"]])
```

Step 12: a row per year

Pivot to a row per year and a column per article. A given cell plots it.

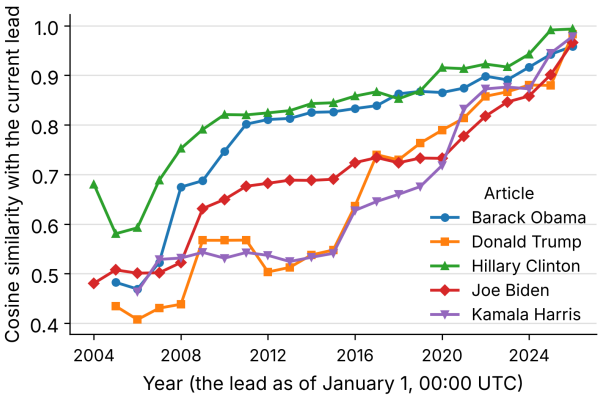
Where do the lines jump? What happened that year?

```
# The January 1 leads, without today's
past = leads[leads["year"] != "current"]
# A row per year, a column per article
table = past.pivot(
    index="year",
    columns="title",
    values="similarity",
)
# Years as numbers, so they sort in order
table.index = table.index.astype(int)
print(table.round(3))
```

Step 12: twenty-three years of leads

Your plot should look like this one.

A similarity says how far a lead's words moved toward today's. It doesn't say why. For that, read the leads.



Microsprint: the biggest jump

Pick an article. `diff()` gives each year's change, and `idxmax()` the year of the biggest rise.

```
title = "Kamala Harris"
# Each year's change from the year before
jumps = table[title].diff()
# The year of the biggest rise, and the rise
year = jumps.idxmax()
print(year, round(jumps.max(), 3))
```

Microsprint: read before and after

Read the lead the year before, and the year of the jump.

Was it a new office, or a rewrite?

```
# That year's lead, and the year before's
for y in [year - 1, year]:
    is_title = leads["title"] == title
    is_year = leads["year"] == str(y)
    text = leads[is_title & is_year]["lead"].iloc[0]
    print(y, text[:400])
```

Step 13: the shape of an answer with sources

Now ask a question, and require the model to name its sources.

This schema is given. `year` may be `null`: no passage says it.

```
# A year (or null), and the IDs of its sources
ANSWER_SCHEMA = {
    "type": "object",
    "properties": {
        "year": {"type": ["integer", "null"]},
        "source_ids": {
            "type": "array",
            "items": {"type": "string"},
        },
    },
    "required": ["year", "source_ids"],
}
```

Step 13: search by meaning

Embed the question as Step 12 embedded a lead. Score every lead against it, and keep that article's best three.

This is **retrieval**: find the passages before you ask.

```
def search(question, title):
    # Embed the question, as Step 12 embedded a lead
    response = requests.post(
        f"{openrouter_base}/embeddings",
        json={"model": EMBEDDER, "input": [question]},
        headers=...,
    ).json()
    query = np.array(response["data"][0]["embedding"])
    # Score every lead; keep this article's best three
    query = query / np.linalg.norm(query)
    leads["score"] = vectors @ query
    best = leads[leads["title"] == title]
    return best.nlargest(3, "score")
```

Step 13: find three leads

Which three Biden leads score highest?

```
question = (
    "In which year did the article first say "
    "Joe Biden is the 46th president of the "
    "United States?"
)
# The three Biden leads closest to the question
top = search(question, "Joe Biden")
print(top[["id", "score"]])
```

Step 13: the passages

Join the three leads into `passages`, each headed by its ID, such as `Joe Biden 2022`.

```
# Each lead headed by its ID, with blank lines between
passages = "\n\n".join(
    f"{row.id}: {row.lead}"
    for row in top.itertuples()
)
```

Friday's body, with the new instruction and schema. The user message holds the question and the passages. The notebook builds it inside the body.

```
# The question, then the three leads
user = f"Question: {question}\n\n{passages}"
body = {
    "models": MODELS,
    "temperature": 0,
    "messages": [
        {"role": "system", "content": ANSWER_INSTRUCTION},
        {"role": "user", "content": user},
    ],
    ... # response_format with ANSWER_SCHEMA; provider
}
```

Step 13: the answer

Send it as on Friday.

Which year did it give, and which sources?

```
# Send it as on Friday
response = requests.post(...).json()
# Which model answered, and its answer
message = response["choices"][0]["message"]
print(response["model"], json.loads(message["content"]))
```

Step 13: check it by reading

Print the start of each Biden lead from 2020 on.

In which year's lead is he the 46th president, not about to become it? Does that match the model's year?

```
# Biden's leads from 2020 on ("current" sorts after)
is_biden = leads["title"] == "Joe Biden"
recent = leads["year"] >= "2020"
# The ID and the start of each
for row in leads[is_biden & recent].itertuples():
    print(row.id, "|", row.lead[:300], "\n")
```

Microsprint: a question with no answer

Ask the same about Kamala Harris as the 47th president.

No lead says it. Did the model answer `null`, or did it name a year?

Why null matters

An answer you can check needs its sources, or an honest `null`.

Step 14: a tool list

A model can't fetch data. It can only write a request. An **MCP server** tells a program which requests it may write.

This given cell loads the Census Bureau's server's tool list, saved on November 9.

```
# The server's tool list, saved on November 9
tools = requests.get(
    cmcp + "tools.json",
    headers=HEADERS,
).json()
# Each tool's name, and the inputs it requires
for tool in tools:
    print(tool["name"], tool["inputSchema"]["required"])
```

Step 14: offer the model the tools

A given cell puts each tool in OpenRouter's form, `functions`, and sets week 11's question.

Send them together. `tool_choice` `"auto"` lets the model pick a tool.

```
body = {
    "models": MODELS,
    "temperature": 0,
    # The tools; the model may choose one
    "tools": functions,
    "tool_choice": "auto",
    "messages": [
        {"role": "system", "content": TOOL_INSTRUCTION},
        {"role": "user", "content": question},
    ],
}
```

Step 14: the model writes one tool call

The model answers with a tool call: a tool's name and its arguments.

Which tool did it pick, and with which arguments?

```
# Send it as on Friday
response = requests.post(...).json()
# The answer is a tool call: a name and arguments
message = response["choices"][0]["message"]
call = message["tool_calls"][0]["function"]
print(response["model"], call["name"], call["arguments"])
```

Step 14: check the call

The model only wrote the call. Our code runs it.

This given cell first reads the arguments, then checks the tool's name.

```
# The arguments, from JSON text to a dictionary
args = json.loads(call["arguments"])
# Is it the one tool our code runs?
print(
    call["name"],
    call["name"] == "fetch-aggregate-data",
)
```

Step 14: turn the call into week 11's request

Then it builds week 11's address from the year and dataset, and the `get` parameter from the table or variables.

```
# Week 11's address, from the year and dataset
url = (
    "https://api.census.gov/data/"
    f"{args['year']}/{args['dataset']}"
)
# A whole table as group(...), or the variables
if "group" in args["get"]:
    get = f"group({args['get']['group']})"
else:
    get = ",".join(args["get"]["variables"])
```

Step 14: the parameters

The parameters week 11 sent, filled in from the model's arguments, with your Census key.

```
# Week 11's parameters, with your key
params = {
    "get": get,
    "for": args["for"],
    "in": args["in"],
    "key": census_key,
}
```

Step 14: run it

Send it. Print the data, never the URL: your key is in it.

Is the median income the same as week 11's?

```
# Send it; print the data, never the URL
data = requests.get(
    url,
    params=params,
    headers=HEADERS,
).json()
print(data)
```

What your code still decides

Which tools run. A model can pick any tool it is shown. Our code runs only one.

Which arguments go out. They decide what your code asks for, with your key.

Whether the answer is right. We know it because week 11 found it.

Microsprint

Ask for the city of Boulder by name, with no FIPS codes. Which tool does the model pick? Can our code run it?

Step 14: a skill is a Markdown file

A **skill** is a file of instructions for a model. Its front matter gives a name and a description, which tell a model when to read the rest.

Which of week 9's tools does this one teach?

```
# A public skill's main file, for PDFs
skill = requests.get(
    "https://raw.githubusercontent.com/anthropics/"
    "skills/main/skills/pdf/SKILL.md",
    headers=HEADERS,
).text
# Its front matter: the first five lines
lines = skill.splitlines()
print("\n".join(lines[:5]))
# Its "### " headings: the tools it teaches
print([line for line in lines if line.startswith("### ")])
```

Step 15: what did we learn?

Answer in the notebook, as comments.

- Which of Datalab's output would you publish?
- Which reports did the model get wrong, and why?
- Which jump could you explain only by reading?
- What will you check your project's model outputs against?

Due Sunday, Nov 22

Chapter 13's Recommended Exercises, from its companion notebook, on a public PDF you choose. Graded on a good-faith attempt.

Key takeaways

1. Call a model **as a function**: one request in, JSON or one tool call out. Your code decides what runs.
2. **Valid JSON can still be wrong.** Check it against something you trust.
3. Datalab **read** some of its output and its model **wrote** the rest.
4. The same request can return a different answer. **Record** the model, the date, and the response.
5. A similarity says how much a text changed. **Read** it to learn why.